



Multi-factor stock trading strategy based on DQN with multi-BiGRU and multi-head ProbSparse self-attention

Wenjie Liu^{1,2,3} · Yuchen Gu¹ · Yebo Ge¹

Accepted: 11 April 2024 / Published online: 22 April 2024
© The Author(s) 2024

Abstract

Reinforcement learning is widely used in financial markets to assist investors in developing trading strategies. However, most existing models primarily focus on simple volume-price factors, and there is a need for further improvement in the returns of stock trading. To address these challenges, a multi-factor stock trading strategy based on Deep Q-Network (DQN) with Multi-layer Bidirectional Gated Recurrent Unit (Multi-BiGRU) and multi-head ProbSparse self-attention is proposed. Our strategy comprehensively characterizes the determinants of stock prices by considering various factors such as financial quality, valuation, and sentiment factors. We first use Light Gradient Boosting Machine (LightGBM) to classify turning points for stock data. Then, in the reinforcement learning strategy, Multi-BiGRU, which holds the bidirectional learning of historical data, is integrated into DQN, aiming to enhance the model's ability to understand the dynamics of the stock market. Moreover, the multi-head ProbSparse self-attention mechanism effectively captures interactions between different factors, providing the model with deeper market insights. We validate our strategy's effectiveness through extensive experimental research on stocks from Chinese and US markets. The results show that our method outperforms both temporal and non-temporal models in terms of stock trading returns. Ablation studies confirm the critical role of LightGBM and multi-head ProbSparse self-attention mechanism. The experiment section also demonstrates the significant advantages of our model through the presentation of box plots and statistical tests. Overall, by fully considering the multi-factor data and the model's feature extraction capabilities, our work is expected to provide investors with more precise trading decision support.

Keywords Deep reinforcement learning · Multi-factor · Multi-BiGRU · Multi-head ProbSparse self-attention · Stock trading strategy

1 Introduction

Financial markets are regarded as a significant driving force in the growth and evolution of the global economy. Investing in the stock market is a prominent research field in the financial industry, which involves high returns as well as high risks. The stock market is affected by a variety of factors, such as the corporate finance [9], exchange rates [22] and public sentiment [3]. With the development of technology, quantitative trading plays an important role in stock market investment. The main motivation to promote the development of quantitative trading is to achieve higher and more stable returns in the investment process.

With the continuous development of machine learning, various neural network models have been applied to stock market prediction tasks, including stock price prediction [7, 17, 25], stock trend prediction [2, 19], portfolio management [1, 31], trading strategies [4, 8, 26, 28, 34], etc. The goal of stock price prediction is to determine the future price of stocks, and prediction models typically combine technical analysis and fundamental analysis with external factors such as market sentiment and news events to improve the accuracy of forecasts. Stock trend prediction involves classifying stock trends into three categories: upward, downward, and sideways trends, this typically involves analyzing historical price data to identify trend patterns and the market factors that support the continuation of these trends. Portfolio management involves asset allocation and risk management to optimize the investment portfolio. Quantitative trading strategies use mathematical and statistical methods to identify opportuni-

✉ Wenjie Liu
wenjiel@163.com

Extended author information available on the last page of the article

ties in the market and design algorithms to automatically execute trades.

In the field of stock trading strategies, various applications of deep learning have shown significant potential and advantages. Fister et al. [8] tried to construct a superior strategy for the daily trading on a portfolio of stocks using Long Short-Term Memory (LSTM), which outperforms traditional trading strategies. And then, a hybrid convolutional recurrent neural network was proposed to construct trading strategies to achieve better returns by Yu and Li [34], where Convolutional Neural Network (CNN) is used to capture local fluctuation features, and LSTM is used to learn the long-term temporal dependencies to improve stock performance prediction. Touzani and Douzi [28] proposed a trading strategy designed for the Moroccan stock market, the decision-making rules for stock trading are based on predictions given by LSTM and Gate Recurrent Unit (GRU), where LSTM and GRU predict short-term and medium-term closing prices, respectively. Soleymani and Paquet [24] developed a portfolio management framework based on the DeepBreath deep reinforcement learning framework, which integrates a restricted stacked autoencoder and a CNN module into a unified framework, they also tested the performance of the DeepBreath framework, achieving investment returns that outperform current expert investment strategies while minimizing market risk.

Deep Q network (DQN) is a type of deep reinforcement learning algorithm that has been widely used to solve various challenging tasks. In recent years, with the development of financial technology, the application of DQN in stock trading strategies has also received widespread attention. In 2020, Lei et al. [13] proposed a Time-driven Feature-aware Jointly Deep Reinforcement Learning model (TFJ-DRL) which is evaluated on real-world financial data with different price trends. In 2021, Ma et al. [18] proposed a novel model named Parallel Multi-Module deep Reinforcement Learning (PMMRL) algorithm, where Fully Connected (FC) layers and LSTM layers are used to extract and encode features of stocks. In 2023, Wang et al. [29] used Light Gradient Boosting Machine (LightGBM) to extract and classify turning points in stock prices, and employed DQN and its related models to train and form a set of reinforcement learning trading strategies. In identical year, Liu et al. [16] proposed a Multi-type data Fusion framework with Deep Reinforcement Learning (MSF-DRL) that integrate stock data, technical indicators, and candlestick charts for algorithmic trading, this approach extracts and fuses temporal features from these data sources to aid in making trading decisions. Although many of the above methods are used to construct trading strategies, the returns achieved are slightly low and the feature extraction ability of the model can be further improved.

Based on the DQN model, we propose a multi-factor stock trading strategy based on DQN with Multi-BiGRU

and multi-head ProbSparse self-attention. The main contributions include three aspects:

- (1) To characterize stock prices from multiple perspectives, a new multi-factor strategy, including financial quality factors, valuation factors, sentiment factor, etc., is adopted, this approach enhances predictive capabilities and adapts to market changes.
- (2) The DQN with Multi-BiGRU is designed, aiming to enhance the model's ability to understand the dynamics of the stock market, ultimately leading to the generation of more effective and robust trading strategies.
- (3) The multi-head ProbSparse self-attention is designed to effectively capture the long-term dependencies between multiple factors and help the model adapt to dynamically changing financial market data.
- (4) Compared with state-of-the-art algorithms in recent years, the proposed model based on DQN with Multi-BiGRU and multi-head ProbSparse self-attention achieves higher returns in individual stocks across Chinese and American markets. This significant advantage is also confirmed by ablation experiments, statistical tests, and the boxplots of the experimental results.

The rest of this paper is organized as follows. First, we give a brief related work on DQN and attention mechanisms in stock trading strategies in Section 2. In Section 3, we introduce LightGBM, BiGRU, DQN, and the stock market terms used in our research. Then, we illustrate the formulation process of the proposed strategy in Section 4. Afterwards, in the Section 5, we make an empirical study on the strategy and discusses its effectiveness. Finally, Section 6 summarizes this paper and discusses the future research direction.

2 Related work

2.1 DQN with non-temporal models

As a typical reinforcement learning algorithm, DQN [21] has been concerned by many scholars, which is widely used in games, robot control, autonomous driving and other fields. In recent years, DQN has become possible in the field of stock trading. Li et al. [14] proposed a novel stock trading strategy consisting of DQN, Double DQN, and Dueling DQN respectively, and found that the DQN model performed best in dealing with stock market strategy decision-making problems. Shi et al. [23] proposed a stock price trend prediction and stock trading model based on Double DQN, and added a well-designed CNN layer to the strategy network to further extract and leverage potential dependencies in stock data, thereby improving model stability. Based on using DQN, Liu et al. [15] took into account the sentiment of investors

in the forecast of the stock market trend, and dynamically adjusted the length of the input market data according to the market conditions. Takara et al. [27] proposed a deep reinforcement learning model called Extended Trading DQN (ETDQN) based on DQN, ETDQN prioritizes experiences that include different sub-goals, allowing the model to accumulate maximum profit without the need for complex reward fine-tuning. Wang et al. [29] used LightGBM to classify turning points in stocks, then used DQN to construct trading strategies, and finally combined the two parts. In these DQN architectures, the Q networks, such as CNNs, automatically extract local features from input data through convolutional layers and pooling layers. However, for stock time series data, their performance in handling long-term dependencies is limited. Therefore, in our selection of Q networks, we have chosen temporal networks. In our comparative experiments, we compared the experimental results with both temporal and non-temporal networks.

2.2 DQN with temporal models

Considering the time series data of investment products such as stocks, many scholars have introduced time series neural networks such as Recurrent Neural Network (RNN) into the DQN model. Wu et al. [32] used GRU to extract financial information characteristics, and proposed two trading strategies with reinforcement learning methods as Gated Deep Q-Network trading strategy (GDQN) and Gated Deterministic Policy Gradient trading strategy (GDPG), then tested stocks from different countries. Huang et al. [11] proposed a novel deep reinforcement learning algorithm called dubbed Two-Branch Deep Q-Network (TBDQN), for generating reliable trading signals for crude oil and natural gas futures. In their another work, Huang et al. [12] proposed a novel algorithmic trading method called CR-DQN, which integrates deep Q-learning with two popular trading rules: Moving Average (MA) and Trading Range Breakout (TRB). The authors have also designed a reward-driven scheme, aiming to capture the intrinsic features of financial data. Lei et al. [13] proposed TFJ-DRL that combines supervised deep learning and reinforcement learning to enhance feature representation learning and decision-making in financial trading, utilizing attention mechanisms and iterative training to optimize investment returns. Ma et al. [18] proposed a novel model called PMMRL algorithm, in which one module employs FC layers to learn the current market state from the market data of the traded stocks and the fundamental data of the issuing companies; another module uses LSTM layers to capture the long-term historical trends of the market. Liu et al. [16] proposed MSF-DRL, in which LSTM, CNN and BiLSTM networks are used to extract the tempo-

ral features of stock data and the candlestick chart. In the above models, for diverse and changing stock data, the feature extraction ability of the model can be further improved. Considering that future prices are influenced by past prices and the importance of trend identification in the stock market, we introduced Multi-BiGRU to better learn the forward and backward dependencies in sequential data and capture long-term dependencies in the sequence data when processing stock multi-factor data.

2.3 Attention mechanism

To capture key information and enable adaptive learning, the attention mechanism has been proposed and is increasingly being used in tasks related to the stock market. Han et al. [10] proposed an Industry Relationship-Driven Hypergraph Attention Network (IRD-HGAT) to predict stock price movement trends. The hypergraph attention mechanism is utilized to dynamically adjust the interactions between stocks, while the features of industry hyperedges are combined to gauge the influence of industry relationships on stock prices. Yang et al. [33] proposed a stock prediction model MDF-DMC that combines multi-view stock data features with dynamic market-related information, multi-head attention mechanisms and position masking matrices are used to capture the relevance between the stock to be predicted and the market. Cui et al. [6] proposed a novel hybrid model named McVCsB, which integrates multi-channel VMD-CBAM-BiLSTM, based on an improved decomposition-reconstruction prediction framework for stock index forecasting, the model utilizes the Variational Modal Decomposition algorithm (VMD) and the Convolutional Block Attention Module (CBAM) to achieve noise filtering and deep-level feature extraction. To effectively capture the important features and their nonlinear relationships among stock multi-factor data, we introduced multi-head ProbSparse self-attention. Multi-head ProbSparse self-attention introduces a probabilistic mechanism to sparsify attention weights, reducing the model's focus on irrelevant information, thereby enhancing efficiency and robustness in processing dynamically changing financial market data.

3 Preliminaries

3.1 LightGBM

Light gradient boosting machine (LightGBM) [20] is a gradient boosting model based on decision trees, which has the advantages of efficient parallel training, feature parallelism, and fast processing of massive data. LightGBM is used for

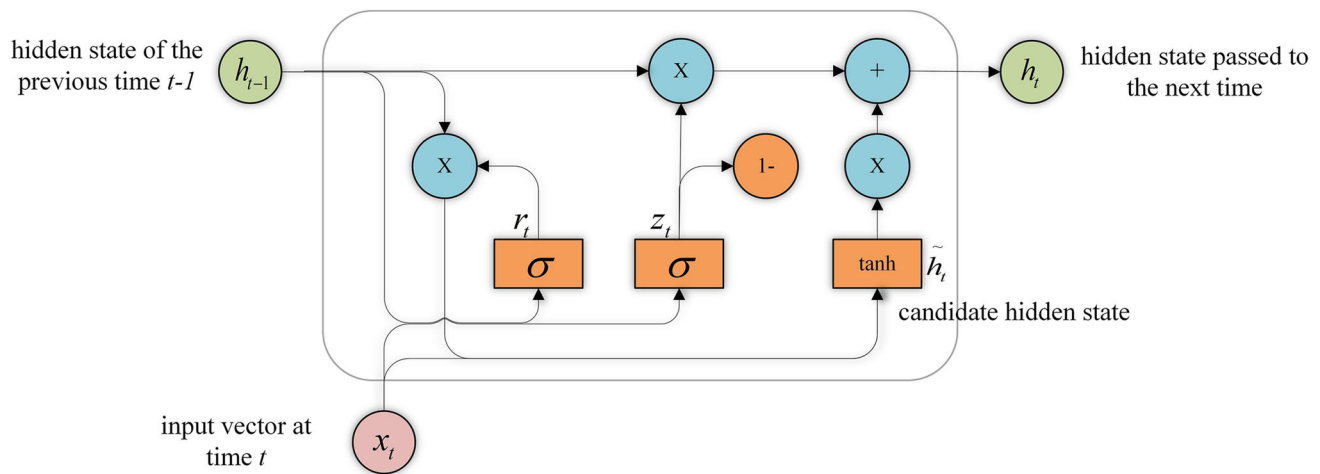


Fig. 1 Structure diagram of GRU

various machine learning tasks such as sorting, classification, and regression. The optimization of LightGBM is as follows:

- (1) Unilateral gradient sampling algorithm is used where only data with high gradients is used when calculating information gain, reducing time overhead.
- (2) Mutually exclusive feature bundling is used which can bind multiple mutually exclusive features into one feature, achieving the goal of dimensionality reduction.
- (3) Histogram algorithm is used to obtain sub optimal solutions for segmented features and segmentation points, and subtracting the histogram of a child node from the histogram of a parent node, the histogram of another child node can be obtained, reducing the computational complexity by half.
- (4) The leaf growth strategy of Leaf-wise with depth limitation is used to split only the points with the maximum information gain, so as to avoid overfitting.

3.2 GRU

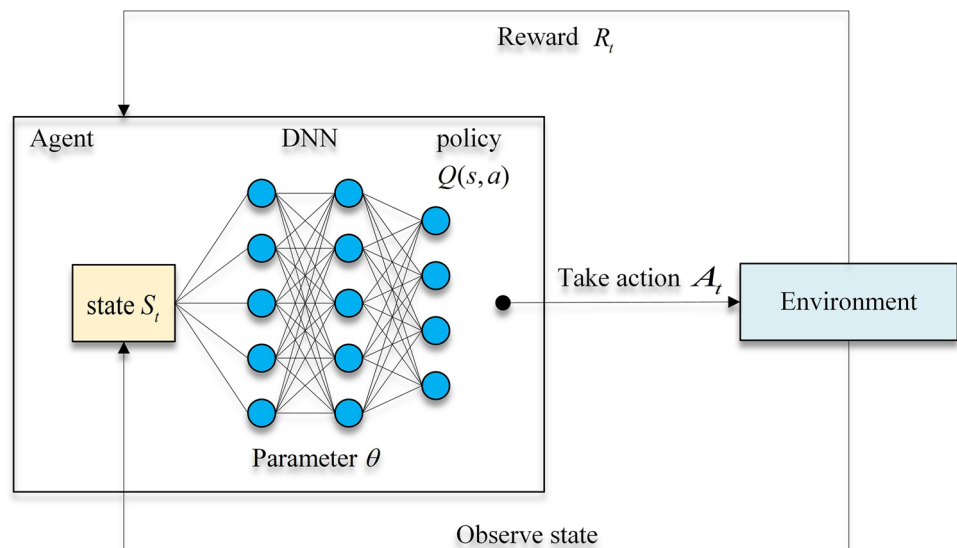
Gated recurrent unit (GRU) [5] is a variant of the LSTM network. It is a very popular sequential neural network. It can solve the long-term dependency problem in the RNN network. The structure of GRU is shown in Fig. 1.

GRU introduces the concepts of reset gate and update gate, the inputs for reset and update (Fig. 2) gates are both the input for the current time step x_t and the hidden state of the previous time step h_{t-1} . The mathematical expression of update and reset gates are as follows:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (1)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (2)$$

Fig. 2 Schematic diagram of DQN



where z_t and r_t are the the update gate and reset gate, W_z and W_r are the weight matrixs, x_t is the input for the current time step, h_{t-1} is the hidden state of the previous time step, and σ represents the sigmoid function.

After calculating the update gate and reset gate, GRU will calculate $\tilde{h}_t$, and the calculation formula for candidate hidden states is as follows:

$$\tilde{h}_t = \tanh(W_{\tilde{h}} \cdot [r_t \odot h_{t-1}, x_t]) \quad (3)$$

Where $\tilde{h}_t$ is the candidate hidden states. The calculation formula for the output h_t of GRU at the last time t is as follows:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (4)$$

3.3 DQN

Reinforcement learning is inspired by behaviorist psychology, focusing on online learning and attempting to keep balance between exploration and utilization. Q-Learning [30] is a value based algorithm in Reinforcement learning, the simplest implementation of Q-learning is to store reward values in a Q-table, but this approach is limited by the number of states and action spaces. To solve this problem, deep neural network was introduced into Q learning by the DeepMind team [21], called deep Q network (DQN), in which deep neural network is used as a function approximator to effectively process high-dimensional data and extract key features.

The network serves as an approximation function to calculate the action values of a series of predefined actions for a given state variable s as a cumulative future reward. The

approximate function is expressed as $Q(s,a)$. When selecting actions based on the network, the actions with the maximum Q value in the current state s are selected as further actions. The target value function for DQN is as follows:

$$y = R_t + \gamma \max_a Q(s', a | \theta') \quad (5)$$

where R_t is the reward obtained after taking action a in the current state, γ is the discount factor, s' is the next state, and θ' is the parameter of the target network.

3.4 Glossary

In the subsequent content of the article, particularly in the section describing multi-factor data, some stock terms and proper nouns are involved. For example, NP represents Net profit, and OI represents Operating income. A complete list of stock terms and their corresponding definitions are provided in Appendix A.

4 Multi-factor stock trading strategy based on DQN with Multi-BiGRU and Multi-head ProbSparse Self-attention

4.1 Overview

The overall framework of the stock trading strategy proposed in this paper is shown in Fig. 3. It can be summarized as follows:

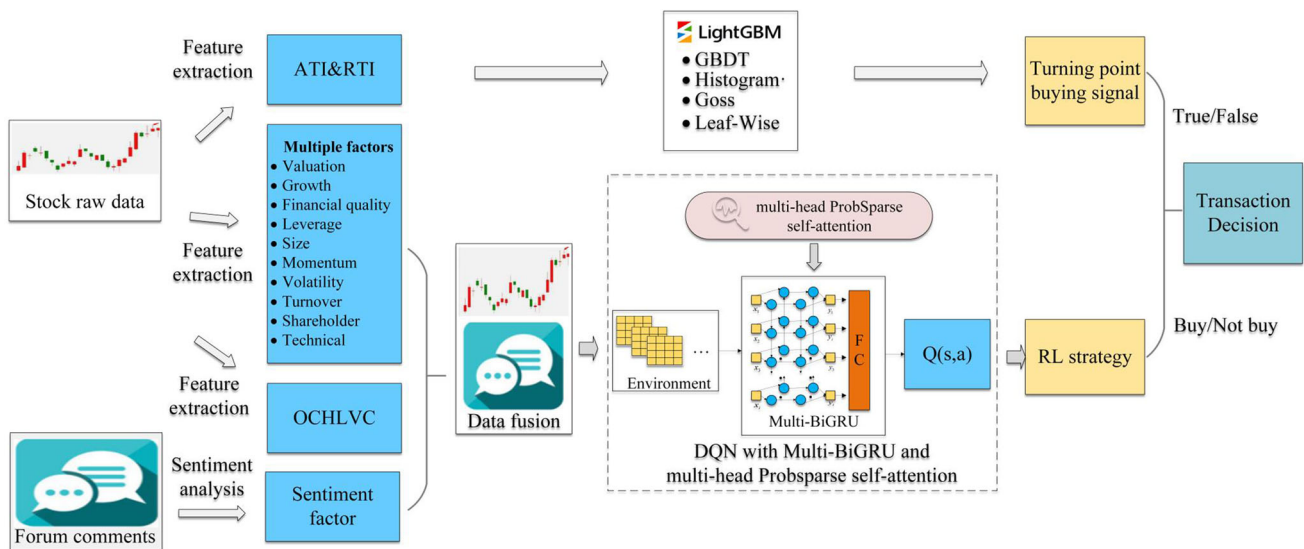


Fig. 3 Process of the proposed stock trading strategy based on DQN with Multi-BiGRU and multi-head ProbSparse self-attention

- (1) **Multiple factors acquisition** We calculate ATI&RTI (absolute technical indicators, relative technical indicators), OCHLVC (open, close, high, low, volume and change) and multiple factors, then the comment text of individual stocks is crawled through forums, and their emotional values are obtained through the BiLSTM model.
- (2) **Data fusion** We fuse multiple factors, OCHLVC and sentiment values and use the z-score method to normalize them.
- (3) **LightGBM model** The LightGBM model is used to obtain the turning point buying signal based on the input ATI&RTI.
- (4) **DQN with Multi-BiGRU and multi-head ProbSparse self-attention**
After receiving multiple factors as input, the trained reinforcement learning model obtains Q values corresponding to different actions to generate reinforcement learning trading strategies.
- (5) **Strategy combination**
If the result of turning point buying signal is true (non-top turning point) and the decision result of RL strategy is 'buy', the transaction decision should be to buy, while the stock will be sold on the next day by default.

$$\text{Transaction decision} = \begin{cases} \text{Buy, non-top turning point and 'buy'} \\ \text{Not buy, top turning point or 'not buy'} \end{cases} \quad (6)$$

4.2 Multiple factors processing and Sentiment analysis

4.2.1 Multiple factors processing

In this paper, we select multiple factors that affect the stock prices. The types of factors are described in Table 1:

After obtaining the basic volume-price factors and financial factors, we can calculate more valuable multiple factors based on the formulas. For example, the turnover rate in the past month can be obtained through volume and outstanding shares (OS).

$$TURNOVER_{1M} = \frac{SUM(Volume, 20)}{OS} \quad (7)$$

Absolute technical indicators, relative technical indicators and complete multiple factors' definitions are shown in Appendix B.

Table 1 Descriptions of factors

Factor Type	Description
Valuation factor	Long term growth expectations for a stock, such as EP, BP, SP, etc.
Growth factor	Profit growth and future development potential of the company, such as the growth rate of net profit, the growth rate of net assets, etc.
Financial quality factor	Composition of multiple financial indicators with different dimensions, such as ROE, gross profit, etc.
Leverage factor	Potential risks and expected returns of stocks, such as current ratio, cash ratio.
Size factor	Company's operating scale, such as the market value.
Momentum factor	Changes and yields in closing prices, such as the past month's yield, etc.
Volatility factor	Stability of stock's rate of return, such as the ratio of the highest and lowest prices, standard deviation of daily yield, etc.
Turnover factor	Frequency of stock turnover and liquidity of stock, such as the turnover rate in the past month, etc.
Shareholder factor	Concentration of stock chips and investment behaviors of both shareholders and retail investors, such as per household shareholding ratio, etc.
Technical factor	Technical indicators for inferring price trends, such as DIF, DEA, etc.

Then we fuse multiple factors, OCHLVC and sentiment values and use the z-score method to normalize them. The normalization formula is as follows.

$$z = \frac{x - \mu}{\sigma} \quad (8)$$

where z is the value after normalization, x represents the value before normalization, μ represents the overall average, and σ represents the overall standard deviation.

4.2.2 Sentiment analysis

We crawled the comment text data of individual stocks over the past decade from economic forums such as Eastmoney, Snowball Finance. The comment text is passed into the BiLSTM model to analyze sentiment values for one of the input

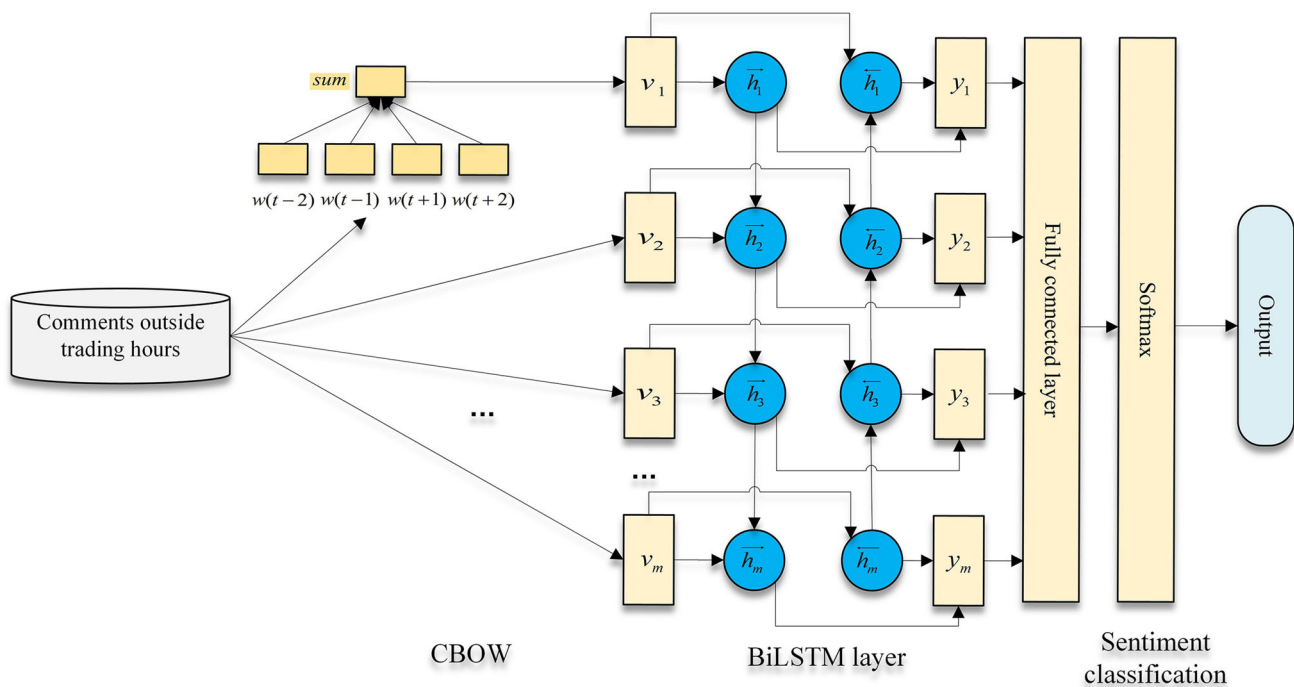


Fig. 4 Process of the sentiment analysis

data of the reinforcement learning model. The model structure is shown in Fig. 4.

(1) Data preprocessing

Considering that sentiment during trading hours is mostly (Fig. 5) reflected in stock price data, comment text outside trading hours is selected.

(2) CBOW

Compared to skipgram, CBOW trains faster and works better for frequently occurring words. We use the CBOW-based training model as the foundation for training word vectors, where w_t represents the t -th word in the text, and $w_t \in R^d$, then the word vector matrix M

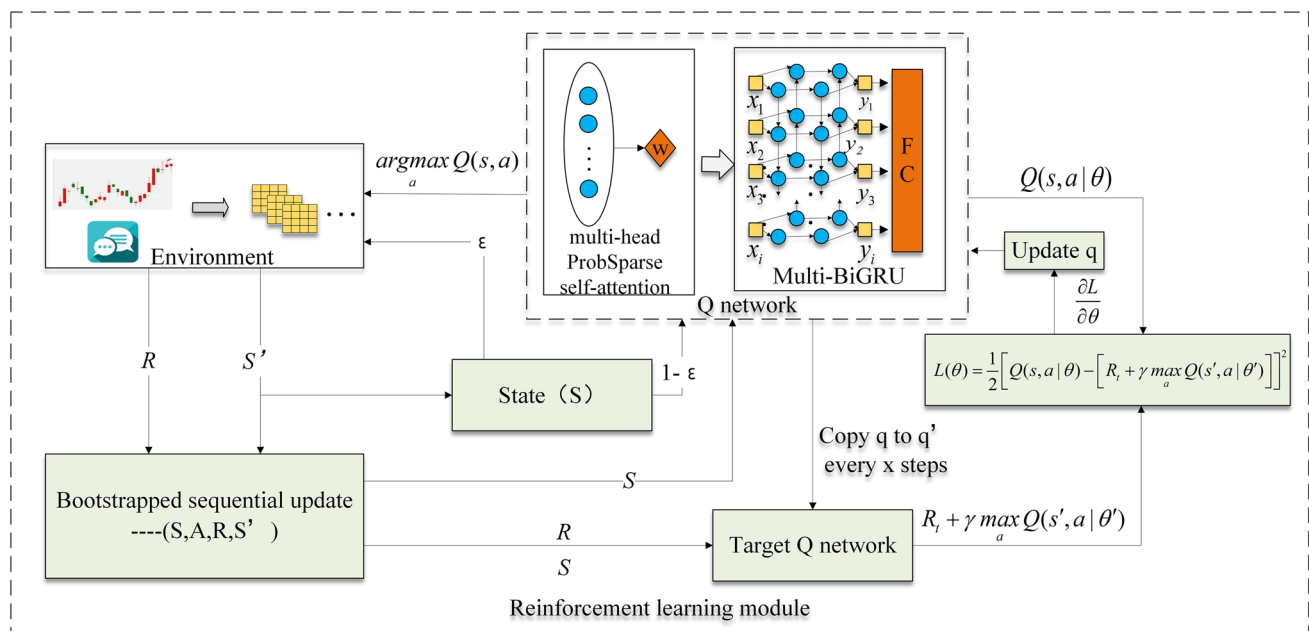


Fig. 5 Model structure of DQN with Multi-BiGRU

can be represented as:

$$M = \{w_1, w_2, w_3, \dots, w_n\} \in R^{n \times d} \quad (9)$$

After text word vectorization, the word vector matrix M is input into the BiLSTM.

(3) BiLSTM layer

BiLSTM is composed of two LSTMs in different directions, which can better capture bidirectional semantic features. The output of BiLSTM at time t concatenates the outputs of forward and backward LSTM at time t using vectors.

$$\vec{h}_t = f(Ux_t + W\vec{h}_{t-1}) \quad (10)$$

$$\overleftarrow{h}_t = f(U'x_t + W'\overleftarrow{h}_{t-1}) \quad (11)$$

$$y_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (12)$$

where $\vec{h}_t$ and $\overleftarrow{h}_t$ are the outputs of forward and backward LSTM, W , U , and V represent the weight matrices for forward computation, and W' , U' , and V' represent the weight matrices for backward computation.

(4) Sentiment classification

Transfer the output to the fully connected layer for sentiment label prediction, and use the Softmax function to obtain the sentence level sentiment probability distribution. The function is defined as:

$$\text{Softmax}(y_t) = \frac{e^{y_t}}{\sum_{c=1}^C e^{y_c}} \quad (13)$$

where y_t is the output value of the t -th node, C is the number of output nodes.

4.3 DQN with Multi-BiGRU

Considering the time-series characteristics of the stock price, the Multi-BiGRU network is integrated into DQN as the Q network, allowing the model to fully consider all information before the current day while observing locally, in order to improve the model's prediction accuracy.

In a single GRU, due to the inconsistency between the length of the input vector and the length of the hidden layer feature values, w is separated by the input and hidden states in each formula. The mathematical expressions for the GRU unit are as follows:

$$r_t = \sigma(w_{ir}x_t + b_{ir} + w_{hr}h_{t-1} + b_{hr}) \quad (14)$$

$$z_t = \sigma(w_{iz}x_t + b_{iz} + w_{hz}h_{t-1} + b_{hz}) \quad (15)$$

$$n_t = \tanh(w_{in}x_t + b_{in} + r_t \odot (w_{hn}h_{t-1} + b_{hn})) \quad (16)$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1} \quad (17)$$

where r_t , z_t and n_t are reset gates, update gates, and status thresholds in GRU, h_t and h_{t-1} are the current hidden state and the previous hidden state.

We combine bidirectional GRU with stacked GRU. In the general case of constructing a network, the hidden state of bidirectional GRU depends on the input value and the previous hidden state; Unlike bidirectional processing, the hidden state of stacked GRUs depends on the previous hidden state and the current hidden state of the previous layer. In Multi-BiGRU, we stack two layers of GRUs in both the forward and backward layers. The mathematical expressions for the first layer BiGRU network structure are as follows:

$$\vec{h}_{1t} = GRU(x_t, \vec{h}_{1t-1}) \quad (18)$$

$$\overleftarrow{h}_{1t} = GRU(x_t, \overleftarrow{h}_{1t-1}) \quad (19)$$

$$h_{1t} = [\vec{h}_{1t}; \overleftarrow{h}_{1t}] \quad (20)$$

where $\vec{h}_{1t}$ and $\overleftarrow{h}_{1t}$ are the forward and backward hidden layer states at time t , $\vec{h}_{1t-1}$ and $\overleftarrow{h}_{1t-1}$ are previous hidden states in the current layer. ';' represents vector concatenation.

In addition to the previous hidden state, the forward and backward networks of the second BiGRU receive the outputs of the first BiGRU's forward and backward networks respectively, and concatenate the forward and backward vectors to complete the information fusion. Finally, they are fed into the linear layer for output mapping. The mathematical expressions for the second layer BiGRU network structure are as follows:

$$\vec{h}_{2t} = GRU(\vec{h}_{1t}, \vec{h}_{2t-1}) \quad (21)$$

$$\overleftarrow{h}_{2t} = GRU(\overleftarrow{h}_{1t}, \overleftarrow{h}_{2t-1}) \quad (22)$$

$$y_t = [\vec{h}_{2t}; \overleftarrow{h}_{2t}] \quad (23)$$

where $\vec{h}_{2t}$ and $\overleftarrow{h}_{2t}$ are the hidden states output by the first BiGRU layer, $\vec{h}_{2t-1}$ and $\overleftarrow{h}_{2t-1}$ are previous hidden states in the current layer.

From the output of the Multi-BiGRU network, we can obtain the estimated output value for the next state. The fitting target of DQN can be seen as a regression problem,

mean square error is used as the loss function to minimize the square loss. The loss function is as follows.

$$L(\theta) = \frac{1}{2} \left[Q(s, a | \theta) - \left[R_t + \gamma \max_a Q(s', a | \theta') \right] \right]^2 \quad (24)$$

where R_t is the reward obtained after taking action a in the current state, γ is the attenuation factor, s' is the next state of s , and $R_t + \gamma \max_{a'} Q(s', a | \theta')$ is the Q value that needs to be optimized.

Because the daily stock price is often associated with the price in the previous days, the serialized update is selected as the update mechanism. After the epoch is randomly selected, it is deduced from the first day of the period T to the last day, then the network parameters are updated with the target network after every three episodes.

The investor's goal is to trade stocks within the investment period T to maximize the cumulative return. For the convenience of subsequent calculations, the logarithm of assets at a certain time can be calculated in the form of compound interest as:

$$\ln(A_t) = \ln(A_0) + \sum_{t=0}^T \ln\left(\frac{close_{t+1}}{close_t}\right) \quad (25)$$

where A_0 represents the initial asset and A_t represents the final asset.

Then the action space for stock trading is defined as buy, not buy. Not buy means being in a short position. The stock will be sold on the next day by default. In network training, in order to avoid the situation that only the same action is selected in the process, the ε -greedy algorithm is used. During the update process, the probability of choosing a random action decreases over time. In this paper, we have set the value of ε to 0.95^t .

$$a = \begin{cases} \arg \max_a Q(s, a), & \text{if } p = 1 - \varepsilon \\ \text{random action}, & \text{if } p = \varepsilon \end{cases} \quad (26)$$

The equation is used at every decision point to select the optimal action, where there is a probability ε of choosing a random action, and a probability of $1 - \varepsilon$ of choosing the action that maximizes the Q-value, i.e., $\arg \max_a Q(s, a)$. During the update process, the probability of choosing a random action decreases over time. Based on the relationship between assets and daily returns, the reward function is set as follows:

$$R_t = \begin{cases} -1.0 * \ln(close_{t+1}/close_t), & \text{action} = 0 \\ 1.0 * \ln(close_{t+1}/close_t), & \text{action} = 1 \end{cases} \quad (27)$$

where action value of 0 indicates not buying on the same day, action value of 1 indicates buying on the same day. At the same time, the reward value that is too large or too small in each transaction should be reduced to prevent excessive fluctuations [29].

$$\overline{R}_t = \begin{cases} R_t * 0.2, & |R_t| \leq 0.01 \\ R_t * 0.7, & 0.01 < |R_t| \leq 0.02 \\ R_t * 1, & 0.02 < |R_t| < 0.07 \\ R_t * 0.95, & |R_t| \geq 0.07 \end{cases} \quad (28)$$

The main purpose of introducing this formula is to smooth out the reward values at each time step, reduce fluctuations, and mitigate excessively high or low reward values. If $|R_t| \leq 0.01$, the reward is set to reward * 0.2; if $0.01 < |R_t| \leq 0.02$, the reward is set to reward * 0.7; and if the value of $|R_t|$ is between 0.02 and 0.07, the reward remains unchanged.

4.4 Multi-head ProbSparse Self-attention mechanism

Stock prices are influenced by many factors, such as fundamental factors, technical factors, market factors, etc. The relationship between these factors is very complex, and it is difficult to effectively capture their interactions using traditional linear models. In 2021, Zhou et al. [35] proposed the informer model to improve the performance of the transformer model, with the proposal attention mechanism improving the traditional attention mechanism. The multi-head ProbSparse self-attention mechanism is capable of simultaneously focusing on multiple factors, capturing important features and their nonlinear relationships, thereby aiming to improve prediction accuracy. Therefore, multi-head ProbSparse self-attention can effectively capture the long-range dependencies between these factors, helping the model better understand the market state.

Multi-head ProbSparse self-attention increases the model's representational power by dividing the attention into multiple heads, with each head learning a different representation subspace. This allows the model to capture a more diverse range of information, which is helpful in generating more precise trading strategies. Additionally, ProbSparse attention introduces a probabilistic mechanism to sparsify attention weights, reducing the model's focus on irrelevant information and thereby improving efficiency and robustness. This is particularly important when dealing with dynamically changing financial market data, as market conditions can shift rapidly, and multi-head ProbSparse self-attention can help the model adapt to these changes. Furthermore, the introduction of multi-head ProbSparse self-attention enhances the model's generalization ability. By learning the intrinsic relationships

between factors, the model can not only capture patterns in historical data but also make more reasonable predictions and decisions in the face of new market conditions. Therefore, we add the multi-head ProbSparse self-attention mechanism to DQN with Multi-BiGRU, allowing the model to handle complex financial market data more flexibly and thereby improve the performance of trading strategies, as shown in Fig. 6.

After vectorizing the multiple factors, the multi-head ProbSparse self-attention mechanism maps the input vector to different subspaces, and each head performs self-attention calculations on different sub regions. In the self-attention mechanism of each head, we obtain three different matrices: query (Q), key (K), and value (V). Subsequently, when calculating the similarity scores between query and key, in traditional attention mechanisms, we need to use dot product to calculate the similarity between each q_i in Q and each k_i in K . However, in a large number of experiments, it can be found that for each q_i , only a small portion of k_i is closely related to it. So in order to filter out closely related q_i and k_i , we calculate the difference between each q_i and a uni-

form distribution. If the difference is large, it indicates that the current q_i is an 'active' query that we need, otherwise it is a 'lazy' query. The expression of the sparsity measure $M(q_i, K)$ of the q_i query is as follows:

$$M(q_i, K) = \ln \sum_{j=1}^{L_K} e^{\frac{q_i k_j^T}{\sqrt{d}}} - \frac{1}{L_K} \sum_{j=1}^{L_K} \frac{q_i k_j^T}{\sqrt{d}} \quad (29)$$

where every q_i, k_i satisfies $q_i \in \mathbb{R}^d, k_i \in \mathbb{R}^d, d$ is the feature dimension in the model and L_K is the length of the key used for attention calculation. In order to reduce computational complexity, a portion of k_i is sampled from the key for calculation, and the q_i and k_i with the highest inner product value are selected to calculate the difference from the uniform distribution. The equation is modified as follows:

$$\bar{M}(q_i, K) = \max_j \left\{ \frac{q_i k_j^T}{\sqrt{d}} \right\} - \frac{1}{L_K} \sum_{j=1}^{L_K} \frac{q_i k_j^T}{\sqrt{d}} \quad (30)$$

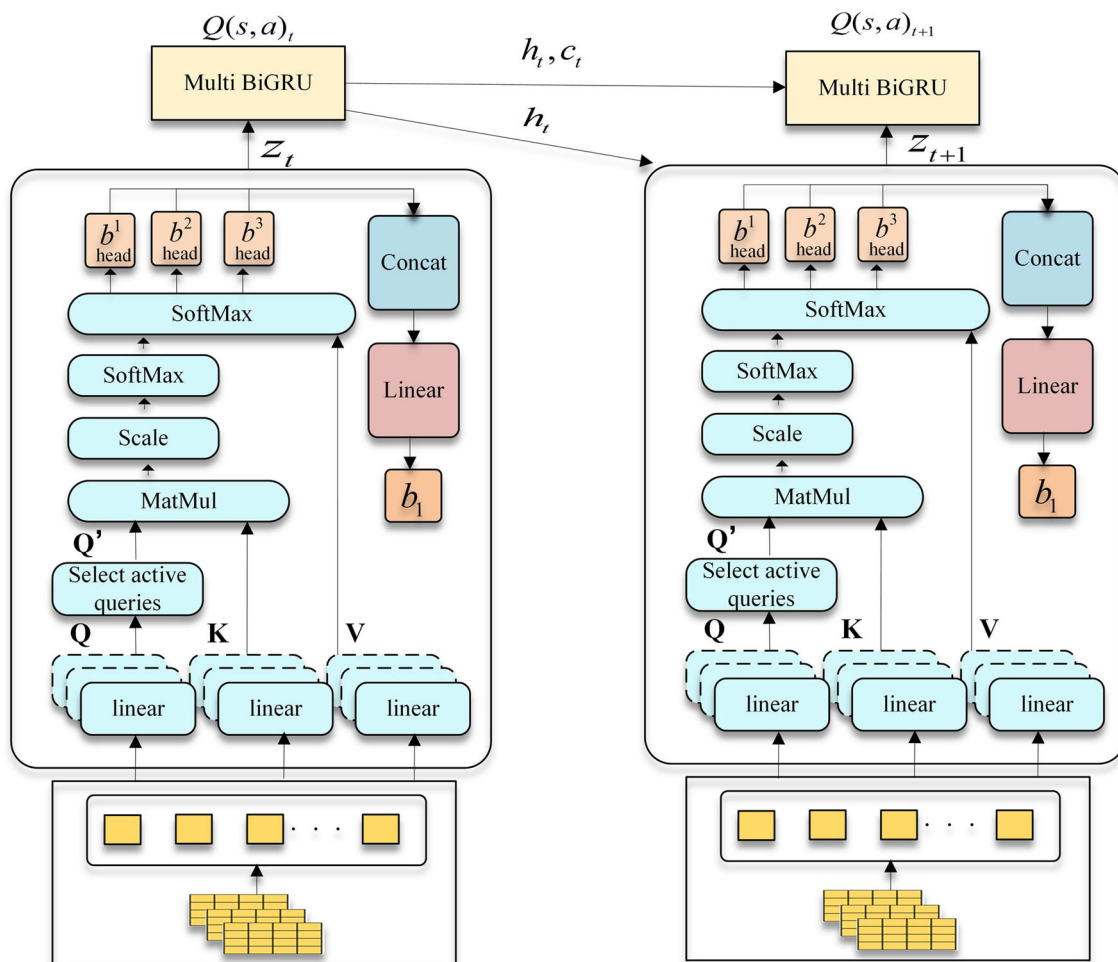


Fig. 6 Structure of Multi-BiGRU with multi-head ProbSparse self-attention mechanism

After calculating the sparsity score, we select the largest n point product results, and use the mean vector as the reconstruction result for the remaining queries. The input and output lengths of the multi-head ProbSparse self-attention are both sequence length. Then the similarity between Q and K is obtained through dot product operation and normalized to $[0,1]$ through softmax, and finally multiplied by the true value V to output the feature matrix. The mathematical representation of attention calculation is given below:

$$Attention(Q, K, V) = softmax(\frac{\bar{Q}K^T}{\sqrt{d_k}}) \cdot V \quad (31)$$

where $\bar{Q}$ is a sparse matrix of the same size as q , d_k represents the dimension of Key and is used for weight scaling. The multi-head ProbSparse self-attention mechanism concatenates the output vectors of each head's ProbSparse self-attention mechanism to obtain a complete output vector, further improving the efficiency and accuracy of obtaining feature importance weights in the sequence. The outputs from every head are concatenated as:

$$Multihead(Q, K, V) = Concat(h_1, h_2, \dots, h_n) \quad (32)$$

$$h_i = Attention(Q \cdot w_i^Q, K \cdot w_i^K, V \cdot w_i^V) \quad (33)$$

Where h_i is the output of the i -th attention head. w_i^Q, w_i^K, w_i^V are weight matrices that determine the degree of matching between queries and keys, as well as how to weight the values based on these matching degrees. In the Multi-BiGRU with multi-head ProbSparse self-attention mechanism, after the feature extraction, the attention mechanism translates the initial vector to the vector z and pass it as input into GRU network to obtain the corresponding Q value. When GRU network is learning, it will output the hidden state h_t and the memory state c_t , where the hidden state will be used as the input of the latter attention mechanism network, and the latter GRU network will consider all

the previous hidden states and memory states together and output the corresponding Q value.

5 Experiment

5.1 Experiment data

Absolute technical indicators and relative technical indicators (ATI&RTI) are used as input data for the LightGBM model for turnover point classification. Multiple factors and sentiment factor are used as inputs for the reinforcement learning model which is trained to form trading strategies. Multiple factors are divided into valuation factors, growth factors, financial quality factors, leverage factors, size factors, momentum factors, volatility factors, turnover factors, shareholder factors, technical factors and emotional factors.

For the convenience of comparison, when comparing with non-temporal models, the selected samples are daily trading data of 40 stocks in the Shanghai and Shenzhen stock exchanges from January 1, 2011 to December 13, 2020. After data fusion and normalization, the data of first 1800 days are used as the training set and the remaining data as the test set. When comparing with temporal models, the selected samples are 30 stocks from both Chinese and US stock markets. The training period ranges from January 1, 2010, to January 1, 2018, and the test period ranges from January 1, 2018, to January 1, 2021. Except for the sentiment value, which requires crawling comments and conducting sentiment analysis, all factors for Chinese stock market stocks are implemented automatically, including technical factors (based on volume and price) and non-technical factors. These are obtained through interfaces in the ITendx quantitative platform using the DLL language (which is widely applicable across many platforms, similar to the PEL language used in the Pyramid platform). The same applies to the technical factors for stocks in the US market. Non-technical factors such as financial data are acquired through third-party libraries like akshare and yfinance, while the remaining few factors

Table 2 Stock samples selected for comparison with non-temporal models

Exchange	Code				
Shanghai stock exchange	600030	600050	600362	601009	601111
	601601	601668	600606	600803	600846
	600737	600155	600111	600016	600056
	601939	600053	600742	600171	600188
Shenzhen stock exchange	000059	002484	000625	000019	000938
	002043	002027	000988	002171	002202
	002251	002302	000065	000878	002145
	000761	000425	000623	000712	300009

Table 3 Stock samples selected for comparison with temporal models

Technology	Financial	Energy	Healthcare	Industrials	Automotive
BIDU	000001	601088	600276	600019	600104
BABA	601939	000983	300015	000708	000625
0700.HK	601288	PTR	ABC	EMR	000996
GOOGLE	BAC	TOT	JNJ	EXPD	TSLA
NVDA	JPM	XOM	MRK	MA	GM

Table 4 The specific parameter settings of DQN with Multi-BiGRU and multi-head ProbSparse self-attention

Parameter	Value	Parameter	Value
hidden_size	128	num_layers	2
learning rate	0.000025	num_heads	8
greedy action probability	0.95 ^t	parameter update method	every 3 epochs
sequence period	250		

Table 5 Profit rate comparison of non-temporal model

Code	DQN(%)	DoubleDQN + LightGBM(%)	DuelingDQN + LightGBM(%)	DQN_CNN + LightGBM(%)	DQN_Multi-BiGRU + LightGBM (%)
600030	77.99	63.63	35.83	101.46	170.24
600050	46.38	33.76	33.76	48.83	61.12
600362	85.45	23.60	4.91	90.44	122.77
601009	23.80	14.17	1.60	24.86	45.43
601111	36.08	117.91	20.35	41.26	50.51
601601	39.29	10.31	18.43	39.29	91.78
601668	34.46	9.09	12.13	38.72	21.45
600606	30.08	34.11	30.50	33.41	43.92
600803	53.88	57.55	16.66	59.38	106.97
600846	86.74	50.90	27.91	100.97	74.66
600737	92.81	24.08	12.51	100.51	112.09
600155	45.96	49.16	55.45	47.75	49.23
600111	19.32	20.58	-5.53	34.32	54.15
600016	9.32	-2.96	-3.94	11.32	24.88
600056	29.96	15.50	6.45	29.96	51.40
601939	1.64	1.85	0.09	2.92	28.08
600053	103.64	72.63	149.43	103.64	28.14
600742	21.36	44.89	4.19	21.36	88.92
600171	159.43	33.50	127.78	163.28	173.81
600188	9.02	5.92	5.87	9.02	22.11
000059	59.80	0.87	7.02	61.58	24.31
002484	45.77	10.03	22.30	38.30	163.96
000625	568.15	291.36	193.17	579.98	266.93
000019	41.90	28.53	113.40	41.90	74.70
000938	18.73	-16.05	22.21	19.35	70.09
002043	185.28	55.60	91.52	198.41	109.40
002027	25.10	29.80	49.61	34.51	97.66
000988	153.60	72.01	122.15	156.71	189.87
002171	46.54	11.19	24.39	46.54	71.55
002202	42.19	17.73	111.04	42.19	87.12

Table 5 continued

Code	DQN(%)	DoubleDQN + LightGBM(%)	DuelingDQN + LightGBM(%)	DQN_CNN + LightGBM(%)	DQN_Multi- BiGRU + LightGBM (%)
002251	3.78	25.89	17.92	3.78	<u>49.76</u>
002302	64.01	34.46	60.92	<u>89.36</u>	34.50
000065	14.88	20.71	-12.85	16.94	<u>47.40</u>
000878	61.06	35.93	52.30	86.35	<u>101.28</u>
002145	41.17	3.72	0.79	47.16	<u>112.17</u>
000761	44.90	12.33	-6.70	<u>49.24</u>	23.89
000425	51.95	13.76	67.15	56.41	<u>80.13</u>
000623	13.13	25.04	9.18	21.51	<u>40.91</u>
000712	203.49	58.37	39.69	<u>212.22</u>	62.41
300009	103.27	19.68	20.43	<u>103.27</u>	72.92

Note: Underlined and bold values represent the best performances

are manually retrieved from websites like Eastmoney and Trading Economics. The stock samples selected for comparison with non-temporal models and temporal models are presented in Tables 2 and 3. Appendix C presents the multi factor (Table 4) data of 40 A-shares as of December 11, 2021.

5.2 Experiment settings

The operating system of the experimental machine is CentOS 8, the CPU is Intel Xeon Gold 5218, the graphics card is NVIDIA GeForce RTX 2080Ti, the code is written in Python language, and the main framework used is PyTorch.

When setting the various network parameters and structures, practical applications often rely on a trial-by-error approach for dynamic adjustment, but there are also certain empirical guidelines that can be followed. We Prevent overfitting through “early stopping” and “regularization,” and use the same number of neurons for all hidden layers. For the learning rate, we first set a very small learning rate, increase the learning rate after each epoch, and record the loss or accuracy for each epoch. The more epochs that are iterated, the more learning rates that are tested. Finally, we compare the loss or accuracy corresponding to different learning rates. The greedy action probability, similarly, decreases over the course of iteration, with the random action probability becoming smaller and smaller. The settings for the greedy action probability and sequence period are the values commonly adopted in most research in this field. Num_layers should not be too large, as it may pose risks such as overfitting and vanishing gradients.

In the DQN with Multi-BiGRU and multi-head ProbSparse self-attention, the following parameters are set: the deep Q network uses Multi-BiGRU, the num_layers is set to 2, the hidden_size is set to 128, the loss function is set to

MSE, and the learning rate is 0.000025. In the multi-head ProbSparse self-attention mechanism, the num_heads is set to 8. The sequence period is 250 days, and the target network is used to update the network parameters after every 3 epochs. The probability of taking random actions in the greedy algorithm is 0.95^t.

5.3 Experiment results

5.3.1 Comparison of non-temporal models

To verify the superiority of our model, we compared the method proposed by Wang et al. [29]. For the convenience of comparison, we selected 40 stocks from Shanghai and Shenzhen Stock Exchanges as experimental samples. The experimental results are shown in Table 5.

From the experimental results in Table 5, it can be seen that DQN+LightGBM with Multi-BiGRU and multi-head ProbSparse self-attention model outperforms DQN+LightGBM in stock tradings. Specifically, the returns achieved by our model are generally higher than those achieved by Buy and Hold. Among the 40 stock samples, our method achieved higher returns than DQN+LightGBM in 80% of the samples. For a small number of samples with poor performance such as 600053 and 000625, we found that most of their amplitudes are relatively large in the test set interval.

Table 6 Friedman test of the results for our model and non-temporal models

P-value	1.45×10^{-13}
Chi-squared value	66.19

Table 7 Nemenyi post-hoc test of the results for our model and non-temporal models

	DQN	DoubleDQN + LightGBM	DuelingDQN + LightGBM	DQN_CNN + LightGBM	DQN_Multi-BiGRU + LightGBM
DQN	1.000	0.124	0.088	0.134	0.001
DoubleDQN + LightGBM	0.124	1.000	0.900	0.001	0.001
DuelingDQN + LightGBM	0.088	0.900	1.000	0.001	0.001
DQN_CNN + LightGBM	0.134	0.001	0.001	1.000	0.481
DQN_Multi-BiGRU + LightGBM	0.001	0.001	0.001	0.481	1.000

After completing the Friedman test for our model and the non-temporal models, as detailed in Table 6, we observed statistically significant differences in the return outcomes between the model we developed and the four comparator baseline models. To further analyze the performance differences between each two models, we conducted a Nemenyi post-hoc analysis. The experimental results showed that the differences in outcomes (Table 7) between our model and DQN, DoubleDQN+LightGBM, and DuelingDQN+LightGBM are statistically significant, but the difference with DQN_CNN+LightGBM is not statistically significant. This is in line with Li's research [14], which suggests that DQN performs better than DoubleDQN and DuelingDQN in terms of stock trading strategies. In order to more intuitively explore the distribution of experimental result data, as shown in Fig. 7, we have plotted box plots for the experimental results,

which reveal that our model provides superior performance in terms of return distribution and median values across a spectrum of individual stocks.

In summary, DQN+LightGBM with Multi-BiGRU and multi-head ProbSparse self-attention model is feasible on the sample dataset, and achieves higher returns than DQN+LightGBM model, it also has shown a significant difference in performance compared to the baseline, with a considerable advantage in terms of returns.

5.3.2 Comparison of temporal models

In order to compare the advantages of our model in algorithms of reinforcement learning with temporal neural network, we selected three models TFJ-DRL [13], PMMRL [18], and MSF-DRL [16] for comparative study. The exper-

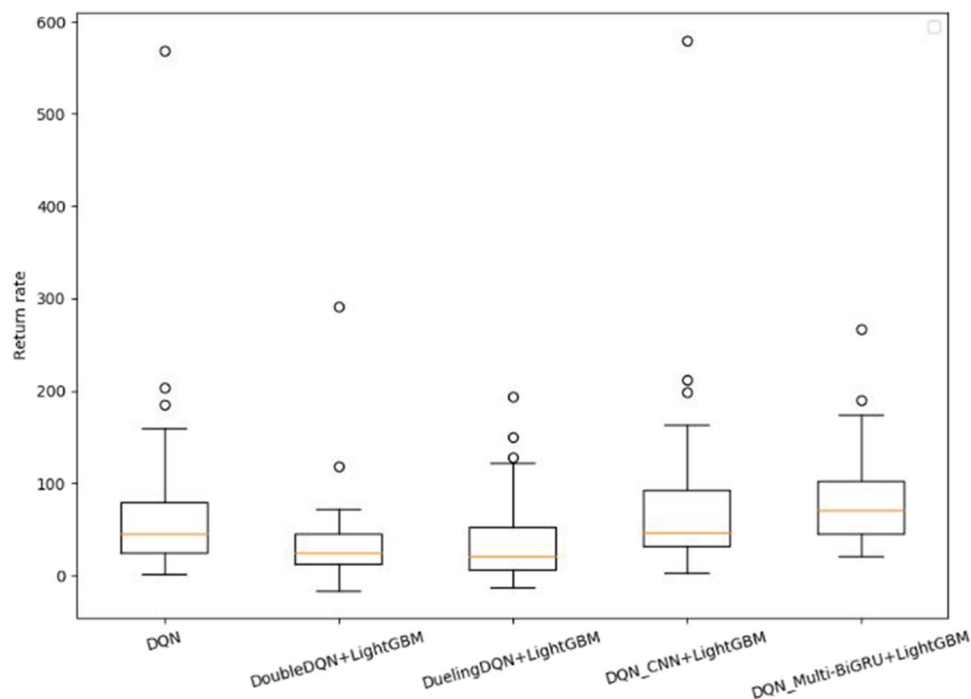
**Fig. 7** Boxplots of our model and non-temporal models' performance

Table 8 Profit rate comparison of RL with temporal model

Sectors	Code	TFJ-DRL(%)	PMMRL(%)	MSF-DRL(%)	DQN_Multi-BiGRU + LightGBM (%)
Technology	BIDU	14.69	15.39	21.40	<u>55.60</u>
	BABA	78.15	83.72	89.26	<u>92.34</u>
	0700.HK	73.90	76.33	<u>81.89</u>	40.96
	GOOGLE	85.73	88.46	101.87	<u>103.93</u>
	NVDA	184.73	197.24	<u>243.56</u>	216.47
Financial	000001	73.52	71.40	79.62	<u>105.81</u>
	601939	-3.82	-2.49	9.06	<u>14.64</u>
	000983	-8.94	-10.42	5.56	<u>9.34</u>
	BAC	37.24	41.64	58.74	<u>59.26</u>
Energy	JPM	18.63	21.41	28.50	<u>37.00</u>
	601088	-2.09	3.48	27.87	<u>29.49</u>
	000983	-8.94	-10.42	5.56	<u>9.34</u>
	PTR	-11.39	-9.36	<u>48.84</u>	14.04
	TOT	-7.29	-5.98	18.26	<u>18.79</u>
Healthcare	XOM	-6.59	-5.93	-1.53	<u>26.96</u>
	600276	98.63	91.47	106.03	<u>159.89</u>
	300015	368.10	410.53	<u>509.99</u>	160.34
	ABC	22.57	23.73	25.75	<u>28.68</u>
	JNJ	30.49	32.64	27.35	<u>43.48</u>
Industrials	MRK	63.02	68.24	65.10	<u>140.14</u>
	600019	-32.57	-36.12	1.21	<u>1.79</u>
	000708	295.91	302.91	<u>371.86</u>	145.30
	EMR	30.78	34.06	39.85	<u>54.87</u>
	EXPD	38.41	40.28	39.59	<u>50.71</u>
Automotive	MA	93.82	95.31	119.79	<u>230.37</u>
	600104	-7.49	-6.84	12.50	<u>46.36</u>
	000625	96.30	90.74	106.41	<u>129.70</u>
	000996	18.36	14.94	37.89	<u>183.58</u>
	TSLA	874.28	973.15	<u>1497.94</u>	221.43
	GM	14.07	16.31	22.63	<u>29.13</u>

Note: Underlined and bold values represent the best performances

imental results are shown in Table 8, where the profit rate of the comparative model is converted into percentage form.

As is shown in Table 8, the samples include six different industries and are composed of stocks in Chinese and US markets. Our proposed method can achieve high returns within the test set and generally outperforms baseline models. Similarly, we can observe from a few underperforming stocks that most of them have significant amplitudes on the test set, such as 0700.HK, PTR, 300015.

Following the Friedman test on the temporal models, Table 9 reveals that the model we introduced exhibits significant differences in terms of return outcomes compared to the four baseline models. To further analyze the perfor-

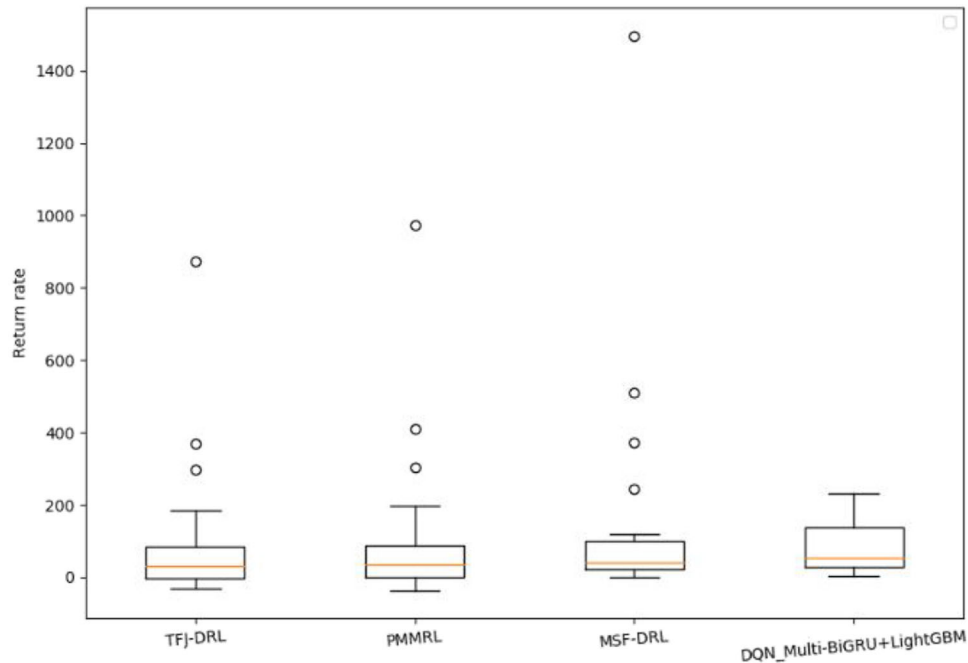
mance differences between each two models, we conducted a Nemenyi post-hoc analysis. The experimental results showed that the differences in outcomes between our model and TFJ-DRL, PMMRL are statistically significant, but the difference (Table 10) with MSF-DRL is not statistically significant. In order to more intuitively explore the distribution of experimental result data, the boxplot in Fig. 8 illustrates that our

Table 9 Friedman test of the results for temporal models

P-value	4.26×10^{-11}
Chi-squared value	51.28

Table 10 Nemenyi post-hoc test of the results for non-temporal models

	TFJ-DRL	PMMRL	MSF-DRL	DQN_Multi-BiGRU + LightGBM
TFJ-DRL	1.000000	0.273433	0.001000	0.001000
PMMRL	0.273433	1.000000	0.007510	0.001000
MSF-DRL	0.001000	0.007510	1.000000	0.499282
DQN_Multi-BiGRU + LightGBM	0.001000	0.001000	0.499282	1.000000

**Fig. 8** Boxplots of temporal models' performance**Table 11** Detailed ablation studies of the stock trading strategy

Code	DQN + Multi-BiGRU(%)	LightGBM + DQN + Multi-BiGRU(%)	LightGBM + DQN + Multi-BiGRU + Multi-head ProbSparse Self-attention (%)
600030	123.47	147.06	170.24
600050	51.90	58.79	61.12
600362	118.06	119.54	122.77
601009	38.48	42.17	45.43
601111	48.22	48.74	50.51
601601	71.38	80.06	91.78
601668	15.45	17.84	21.45
600606	36.92	37.19	43.92
600803	87.19	92.52	106.97
600846	102.04	99.04	74.66
600737	102.27	110.94	112.09
600155	48.72	47.99	49.23
600111	46.25	51.07	54.15

Table 11 continued

Code	DQN + Multi-BiGRU(%)	LightGBM + DQN + Multi-BiGRU(%)	LightGBM + DQN + Multi-BiGRU + Multi-head ProbSparse Self-attention (%)
600016	20.37	22.78	<u>24.88</u>
600056	40.23	40.19	<u>51.40</u>
601939	16.11	22.12	<u>28.08</u>
600053	12.15	14.82	<u>28.14</u>
600742	59.90	48.92	<u>88.92</u>
600171	145.13	164.59	<u>173.81</u>
600188	10.80	17.15	<u>22.11</u>
000059	<u>25.85</u>	23.11	24.31
002484	107.95	147.81	<u>163.96</u>
000625	246.19	249.38	<u>266.93</u>
000019	65.11	65.76	<u>74.70</u>
000938	56.81	65.36	<u>70.09</u>
002043	103.66	104.28	<u>109.40</u>
002027	64.32	86.04	<u>97.66</u>
000988	122.98	132.86	<u>189.87</u>
002171	61.35	65.93	<u>71.55</u>
002202	72.34	78.77	<u>87.12</u>
002251	34.43	34.93	<u>49.76</u>
002302	84.88	<u>85.59</u>	34.50
000065	38.58	39.46	<u>47.40</u>
000878	59.75	70.58	<u>101.28</u>
002145	60.96	66.44	<u>112.17</u>
000761	14.62	15.04	<u>23.89</u>
000425	72.35	75.21	<u>80.13</u>
000623	32.46	33.14	<u>40.91</u>
000712	33.43	44.80	<u>62.41</u>
300009	<u>103.50</u>	97.88	72.92

Note: Underlined and bold values represent the best performances

model outperforms the others in terms of return distribution and median performance across various individual stocks.

5.3.3 Ablation studies

In this section, we conduct ablation studies on the 40 stocks from Shanghai Stock Exchange and Shenzhen Stock Exchange. The LightGBM model, Multi-BiGRU model and multi-head ProbSparse self-attention mechanism are investigated for the effectiveness of each module in more detail. Table 11 provides the ablation results of three configurations.

As is shown in Table 11, we first evaluate the effectiveness of LightGBM turning point classification on the experimental results, and find that LightGBM turning point classification can reduce some wrong decisions in reinforcement learn-

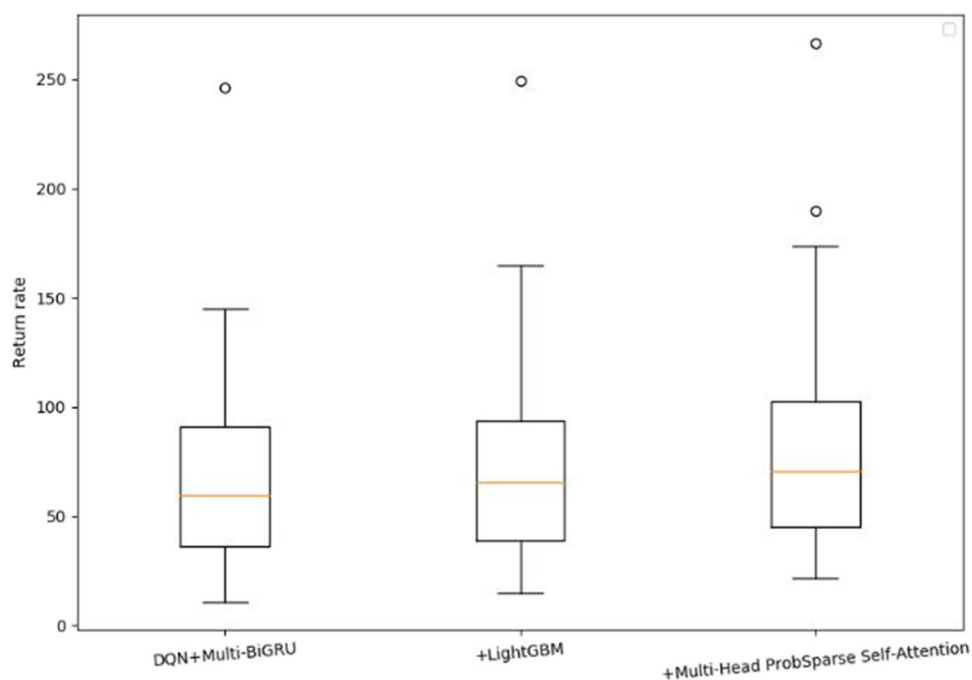
ing to improve the returns achieved; we further evaluated the effectiveness of the multi-head ProbSparse self-attention mechanism on the results and found that it can better extract features from multiple factors to improve the returns of stock trading.

To better analyze the ablation experiment, we conducted a Friedman test on its results. As shown in Table 12, there are significant differences between the proposed model and the model after removing modules. To further observe the

Table 12 Friedman test for the performance in ablation studies

P-value	1.42×10^{-11}
Chi-squared value	49.95

Fig. 9 Boxplots of the performance in ablation studies



performance (Tables 13, 14 and 15) differences between each pair of models, we conducted a Nemenyi post-hoc test. The results reflected that each addition or removal (Tables 16 and 17) of a module had a significant impact on the outcome. From the box plot in Fig. 9, it can be seen that the return distribution and median of our model steadily increased after adding LightGBM and multi-head ProbSparse self-attention, demonstrating the effectiveness of each module in the model.

6 Conclusion

In this paper, we propose a multi-factor stock trading strategy based on DQN with Multi-BiGRU and multi-head ProbSparse self-attention mechanism. In addition to considering the basic data of absolute and relative technical indicators, we also introduce factors such as valuation, financial quality, and sentiment values. These multi-factor data more comprehensively characterize the determinants of stock prices, and the Multi-BiGRU and multi-head ProbSparse self-attention mechanism are also capable of effectively extracting features from multi-factor time series data. In comparative experi-

ments, we compare our model with the latest non-temporal and temporal models, with samples from different individual stocks in Chinese and US stock markets. Our model achieves better returns compared to other models. In ablation experiments, we demonstrate the necessity of each module in the proposed model. To further analyze the experimental results, we have drawn box plots and conducted Friedman test and Nemenyi post-hoc test and discussed the experimental results.

For future work, we consider the following two aspects: 1) there are still many uncertainties in stock data. How to reduce the uncertainty in stock data is a problem worth considering. 2) Risk control in stock investment is very important. Therefore, how to better balance returns and risks in stock trading strategies based on reinforcement learning is a topic worth researching.

Appendix A Glossary

The following table shows the stock terms in the article and their corresponding definitions.

Table 13 Nemenyi post-hoc test for the performance in ablation studies

	DQN + Multi-BiGRU	+LightGBM	+Multi-head ProbSparse Self-attention
DQN + Multi-BiGRU	1.000000	0.007169	0.001
+LightGBM	0.007169	1.000000	0.001
+Multi-head ProbSparse Self-attention	0.001000	0.001000	1.000

Table 14 Glossary

Term	Definition
<i>_TTM</i>	Trailing twelve months
<i>NP</i>	Net profit
<i>EPS_Excl.</i>	Earnings per share after deducting non-recurring main business
<i>NP_Excl.</i>	Net profit after deducting non-recurring gains and losses
<i>NA</i>	Net assets
<i>OI</i>	Operating income
<i>OCF</i>	Operating cash flow
<i>OP</i>	Operating profit
<i>GMV</i>	Gross market value
<i>SR</i>	Sales revenue
<i>CS</i>	cost of sales
<i>TA</i>	Total assets
<i>OCF</i>	Operating cash flow
<i>MF</i>	monetary fund
<i>MS</i>	marketable securities
<i>CL</i>	Current liability
<i>CA</i>	Current assets
<i>OS</i>	Outstanding Shares

Appendix B Multiple factors

The following tables show descriptions of absolute technical indicators, relative technical indicators and complete multiple factors.

Table 15 Absolute technical indicators and relative technical indicators

Factor Type	Factor name	Definition
Absolute technical indicators	Moving average	$SUM(Close, 5)/5$
	Parabolic SAR	$SAR_{n-1} + AF_n * [EP_{n-1} - SAR_{n-1}]$
	ROC	$(Close - REF(Close, 12))/REF(Close, 12)$
	ADX	$MA(DMI)/TR$
	CCI	$(TP - MA)/MD * 0.015$
	MACD	$2 * (DIF - DEA)$
	MFI	$100 - \frac{100}{1+MR}$
	RSI	$100 - \frac{100}{1+RS}$
	Change	$Close_t - Close_{t-1}$
	<i>Pct_change</i>	$\frac{Close_t - Close_{t-1}}{Close_{t-1}}$
Relative technical indicators	<i>OO₁</i>	$\frac{Open_t - Open_{t-1}}{Open_{t-1}}$
	<i>OC₁</i>	$\frac{Open_t - Close_{t-1}}{Close_{t-1}}$
	<i>CH₁₀</i>	$\frac{Close_t - \max(Close_{t-1}, Close_{t-2}, \dots, Close_{t-10})}{Close_t}$
	<i>CL₁₀</i>	$\frac{Close_t - \min(Close_{t-1}, Close_{t-2}, \dots, Close_{t-10})}{Close_t}$

Table 16 Multiple factor data

Factor Type	Factor name	Definition
Valuation factor	EP	NP_TTM/GMV
	EPSCUT	$NP_Excl./GMV$
	BP	NA/GMV
	SP	OI/GMV
	OCFP	OCF/GMV
	SALE_RATE	$OI/REF(OI, 250)$
	PROFIT_RATE	$NP/REF(NP, 250)$
Growth factor	ASSET_RATE	$NA/REF(NA, 250)$
	BIPROFIT_RATE	$OP/REF(OP, 250)$
	EPSCUT_Q	$EPS_Excl./REF(EPS_Excl., 250)$
	PROFITCUT_Q	$NP_Excl./REF(NP_Excl., 250)$
Financial quality factor	ROE	NP/NA
	GROSS_PROFIT	$(SR - CS)/SR$
	PROFITCUT	$NP_Excl.$
	ASSET_TURNOVER	SR/TA
	O_CASHFLOWRATIO	OCF/NP
Leverage factor	CASHRATION	$(MF + MS)/CL$
	CURRENTRATION	CA/CL
Size factor	IN_CAPITAL	$ln(marketvalue)$
Momentum factor	RELATIVE_1M	$Close/REF(Close, 20)$
	RELATIVE_2M	$Close/REF(Close, 40)$
	RELATIVE_3M	$Close/REF(Close, 60)$
	RELATIVE_6M	$Close/REF(Close, 120)$
	RELATIVE_12M	$Close/REF(Close, 250)$
Volatility factor	HIGH_LOW_1M	$HHV(High, 20)/LLV(Low, 20)$
	HIGH_LOW_2M	$HHV(High, 40)/LLV(Low, 40)$
	HIGH_LOW_3M	$HHV(High, 60)/LLV(Low, 60)$
	HIGH_LOW_6M	$HHV(High, 120)/LLV(Low, 120)$
	HIGH_LOW_12M	$HHV(High, 250)/LLV(Low, 250)$
	STD_1M	$STD(Close - Open, 20)$
	STD_2M	$STD(Close - Open, 40)$

Table 16 continued

Factor Type	Factor name	Definition
Turnover factor	STD_3M	$STD(Close - Open, 60)$
	STD_6M	$STD(Close - Open, 120)$
	STD_12M	$STD(Close - Open, 250)$
	LN_PRICE	$\ln(Close)$
	TURNOVER_1M	$SUM(Volume, 20) / OS$
	TURNOVER_2M	$SUM(Volume, 40) / OS$
	TURNOVER_3M	$SUM(Volume, 60) / OS$
	TURNOVER_6M	$SUM(Volume, 120) / OS$
Shareholder factor	TURNOVER_12M	$SUM(Volume, 250) / OS$
	HOLDER_AVGPCT	$\frac{OS}{Number of public shareholders}$
	HOLDER_H_CHANGE	$\frac{HOLDER_AVGPCT}{REF(HOLDER_AVGPCT, 120)}$
Technical factor	HOLDER_Y_CHANGE	$\frac{HOLDER_AVGPCT}{REF(HOLDER_AVGPCT, 250)}$
	EMA12	$REF(EMA12, 1) * \frac{11}{13}$
		$+ Close * \frac{2}{13}$
	EMA26	$REF(EMA26, 1) * \frac{25}{27}$
		$+ Close * \frac{2}{27}$
	DIF	$EMA(12) - EMA(26)$
	DEA	$(DIF * 2 + REF(DEA, 1) * 8) / 10$

Appendix C Multi-factor data of 40 A-shares

The following table shows the multi-factor data of 40 A-shares on December 11, 2020.

Table 17 Multi-factor data of 40 A-shares on December 11, 2020

Code	EP	EPSCUT	BP	SP	OCFP	SALE_	PROFIT_	ASSET_	BIPROFIT_	EPSCUT_	ROE	IN_	RELATIVE_	HIGH_LOW_	STD_	RSI	...
						RATE	RATE	RATE	RATE	Q		CAPITAL	IM	IM	IM		
000059	-0.01	-0.02	1.6	2.39	-0.11	-28.74	-118.14	-3.82	-111.01	-120.68	-1.27	22.87	0.95	1.18	0.12	37.55	
002484	0.03	0.02	0.42	0.2	0.03	17.88	30.07	4.86	12.84	47.01	6.14	22.91	0.96	1.15	0.29	41.87	
000625	0.03	-0.02	0.43	0.5	0.06	23.77	230.98	8.05	226.31	40.63	7.33	25.43	1.35	1.59	1.46	53.77	
000019	0.04	0.03	0.53	0.89	0.04	6.11	-5.75	1.81	-13.33	-9.36	6.9	22.91	1.0	1.08	0.1	40.24	
000938	0.06	0.02	0.58	0.7	0.04	8.98	4.32	3.38	17.26	-13.15	4.45	24.8	0.92	1.14	0.37	29.9	
002043	0.05	0.04	0.27	0.57	0.02	19.66	-23.81	-10.65	-14.96	8.48	11.71	22.62	0.88	1.2	0.15	30.05	
002027	0.02	0.01	0.11	0.06	0.02	-11.57	61.9	10.15	61.65	119.58	14.51	25.65	0.99	1.18	0.32	43.93	
000988	0.02	0.01	0.28	0.19	0.0	-16.3	5.2	6.94	4.56	26.58	7.41	23.87	0.88	1.18	0.33	31.5	
002171	0.03	0.01	0.49	1.33	-0.07	34.35	-44.37	4.03	-41.57	-41.85	3.48	23.19	0.93	1.17	0.21	36.02	
002202	0.05	0.04	0.7	0.75	0.05	49.71	-30.07	8.2	29.72	29.86	6.23	24.62	1.0	1.18	0.33	43.68	
002251	0.01	0.02	0.92	1.81	0.13	1.74	-27.18	-2.06	-28.54	-36.82	2.29	22.84	0.92	1.18	0.18	32.74	
002302	0.07	0.05	0.85	1.39	-0.17	-2.62	6.28	7.71	18.61	3.77	8.44	23.18	0.98	1.09	0.11	39.56	
000065	0.13	0.09	1.07	1.25	-0.23	6.57	3.74	9.3	-0.34	1.66	9.41	22.46	0.96	1.1	0.11	32.96	
000878	0.04	0.02	0.51	2.57	0.12	48.15	-27.72	7.08	2.21	-15.71	4.97	23.97	1.01	1.17	0.47	47.37	
002145	0.04	0.03	0.49	0.23	0.03	6.1	8.29	74.05	7.04	-20.73	6.24	23.18	0.91	1.21	0.15	38.7	
000761	0.03	0.02	1.7	2.76	-0.11	-9.58	-41.84	7.48	-40.92	-48.94	1.35	23.26	1.0	1.06	0.04	40.11	
000425	0.07	0.06	0.76	1.18	0.02	18.61	-19.4	-2.27	-16.43	-11.23	7.51	24.5	1.08	1.22	0.12	52.8	
000623	0.09	0.08	1.21	0.08	0.02	-31.48	29.66	4.38	25.24	32.33	6.47	23.69	0.96	1.08	0.16	34.13	
000712	0.02	0.01	0.41	0.12	0.08	31.3	62.25	5.61	53.32	285.04	5.72	23.33	1.1	1.2	0.39	55.15	
300009	0.01	0.01	0.14	0.06	0.01	-0.91	16.4	5.66	20.79	-14.6	10.78	23.72	0.88	1.17	0.32	32.58	
600030	0.04	0.03	0.49	0.11	0.17	28.13	20.32	11.21	17.53	12.44	7.04	26.64	1.0	1.11	0.44	39.3	
600050	0.09	0.03	2.28	1.56	0.53	3.79	10.84	2.51	15.67	24.72	3.26	25.7	0.98	1.08	0.03	32.05	
600362	0.03	0.02	0.86	3.3	0.1	32.73	-25.8	0.1	-9.56	-34.24	2.9	24.96	1.25	1.42	0.71	55.66	
601009	0.16	0.12	1.3	0.31	0.06	2.42	2.01	19.33	2.86	-9.48	10.75	25.11	1.01	1.14	0.13	41.25	
601111	-0.01	-0.09	0.78	0.43	-0.02	-52.99	-249.54	-13.0	-247.11	-257.78	-12.43	25.45	1.04	1.14	0.18	50.02	
601601	0.07	0.06	0.59	0.97	0.23	9.86	-14.25	13.63	5.07	2.08	9.69	26.59	1.16	1.28	0.81	54.68	
601668	0.31	0.14	2.56	5.03	-0.41	10.53	3.86	4.54	9.83	5.72	10.74	26.09	0.97	1.1	0.06	36.4	
600606	0.3	0.16	1.79	4.34	0.07	9.14	1.94	2.36	1.75	16.15	14.91	25.03	0.98	1.1	0.1	35.31	
600803	0.01	0.01	0.91	1.66	0.22	-10.2	-45.08	-57.51	-6.19	-82.42	17.64	24.3	0.98	1.12	0.23	47.79	
600846	0.22	0.08	0.78	0.88	0.19	8.7	26.11	7.84	23.31	27.61	13.51	22.36	0.99	1.06	0.06	33.1	
600737	0.04	0.04	0.48	0.78	0.18	16.08	28.57	7.32	32.06	25.77	8.75	23.64	1.04	1.14	0.16	42.97	
600155	0.02	0.02	0.67	0.01	0.06	23.71	40.57	0.16	34.82	35.79	3.04	23.83	1.09	1.29	0.47	49.37	

Table 17 continued

Code	EP	EPSCUT	BP	SP	OCFP	SALE_ RATE	PROFIT_ RATE	ASSET_ RATE	BIPROFIT_ RATE	EPSCUT_ Q	ROE	IN_ CAPITAL	RELATIVE_ IM	HIGH_LOW_ IM	STD_ IM	RSI	...
600111	0.02	0.01	0.31	0.36	0.01	10.26	10.13	5.08	14.11	17.63	5.04	24.49	1.01	1.26	0.41	41.07	
600016	0.21	0.16	2.4	0.63	-0.42	7.66	-18.01	2.68	-21.98	0.0	7.01	26.15	1.01	1.07	0.06	42.23	
600056	0.09	0.08	0.78	1.93	-0.04	16.3	27.8	10.49	21.95	29.69	12.46	23.48	0.98	1.1	0.18	37.52	
601939	0.15	0.13	1.45	0.35	0.38	5.89	-8.66	4.79	-10.16	-8.56	9.35	28.12	1.01	1.19	0.12	40.67	
600053	0.08	0.01	0.31	0.03	-0.02	-51.93	-42.32	4.58	-35.27	-42.36	4.74	22.91	0.89	1.21	0.46	22.56	
600742	0.13	0.06	1.12	2.0	0.28	41.14	14.37	10.98	37.85	-15.23	6.68	22.64	0.9	1.22	0.25	31.02	
600171	0.02	0.01	0.28	0.09	0.0	13.74	-23.81	-6.79	-22.35	22.21	4.67	23.05	0.95	1.13	0.31	36.33	
600188	0.18	0.07	1.53	2.96	0.12	2.35	-18.52	-2.93	-23.0	-45.04	9.26	24.67	1.13	1.26	0.29	52.73	

Acknowledgements This work is supported by the National Natural Science Foundation of China (62071240), the Innovation Program for Quantum Science and Technology (2021ZD0302901), the Natural Science Foundation of Jiangsu Province (BK20220804, BK20231142), and the Priority Academic Program Development of Jiangsu Higher Education Institutions (PAPD).

Author Contributions **WenJie Liu:** Conceptualization, Data curation, Formal analysis, Methodology, Project administration, Investigation, Software, Supervision, Funding acquisition, Writing - original draft, Writing - review & editing. **YuChen Gu:** Methodology, Data curation, Formal analysis, Software, Investigation, Visualization, Resources, Writing - original draft. **YeBo Ge:** Formal analysis, Investigation, Validation, Writing - review & editing. All authors read and approved the final manuscript.

Data Availability Data will be made available on request.

Declarations

Competing interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical and informed consent for data used This article does not contain any studies with human participants or animals performed by any of the authors.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Almahdi S, Yang SY (2017) An adaptive portfolio trading system: A risk-return portfolio optimization using recurrent reinforcement learning with expected maximum drawdown. *Expert Syst Appl* 87:267–279. <https://doi.org/10.1016/j.eswa.2017.06.023>
- Aseeri AO (2023) Effective short-term forecasts of Saudi stock price trends using technical indicators and large-scale multivariate time series. *PeerJ Comput Sci* 9:e1205. <https://doi.org/10.7717/peerj-cs.1205>
- Bollen J, Mao H, Zeng X (2011) Twitter mood predicts the stock market. *J Comput Sci* 2(1):1–8. <https://doi.org/10.1016/j.jocs.2010.12.007>
- Chakole JB, Kolhe MS, Mahapurush GD et al (2021) A q-learning agent for automated trading in equity stock markets. *Expert Syst Appl* 163:113761. <https://doi.org/10.1016/j.eswa.2020.113761>
- Cho K, van Merriënboer B, Gulcehre C et al (2014) Learning phrase representations using RNN encoder-decoder for statistical machine translation. In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp 1724–1734. <https://doi.org/10.3115/v1/D14-1179>
- Cui C, Wang P, Li Y et al (2023) Mcvcsb: A new hybrid deep learning network for stock index prediction. *Expert Syst Appl* 232. <https://doi.org/10.1016/j.eswa.2023.120902>
- Deng C, Huang Y, Hasan N et al (2022) Multi-step-ahead stock price index forecasting using long short-term memory model with multivariate empirical mode decomposition. *Inf Sci* 607:297–321. <https://doi.org/10.1016/j.ins.2022.05.088>
- Fister D, Perc M, Jagric T (2021) Two robust long short-term memory frameworks for trading stocks. *Appl Intell* 51(10):7177–7195. <https://doi.org/10.1007/s10489-021-02249-x>
- Hagenau M, Liebmann M, Neumann D (2013) Automated news reading: Stock price prediction based on financial news using context-capturing features. *Decision Support Syst* 55(3):685–697. <https://doi.org/10.1016/j.dss.2013.02.006>
- Han H, Xie L, Chen S et al (2023) Stock trend prediction based on industry relationships driven hypergraph attention networks. *Appl Intell*. <https://doi.org/10.1007/s10489-023-05035-z>
- Huang Z, Gong W, Duan J (2023) Tbdqn: A novel two-branch deep q-network for crude oil and natural gas futures trading. *Appl Energy* 347. <https://doi.org/10.1016/j.apenergy.2023.121321>
- Huang Z, Li N, Mei W et al (2023) Algorithmic trading using combinational rule vector and deep reinforcement learning. *Appl Soft Comput* 147. <https://doi.org/10.1016/j.asoc.2023.110802>
- Lei K, Zhang B, Li Y et al (2020) Time-driven feature-aware jointly deep reinforcement learning for financial signal representation and algorithmic trading. *Expert Syst Appl* 140:112872. <https://doi.org/10.1016/j.eswa.2019.112872>
- Li Y, Ni P, Chang V (2020) Application of deep reinforcement learning in stock trading strategies and stock forecasting. *Computing* 102(6, SI):1305–1322. <https://doi.org/10.1007/s00607-019-00773-w>
- Liu C, Yan J, Guo F et al (2022) Forecasting the market with machine learning algorithms: An application of NMC-BERT-LSTM-DQN-X algorithm in quantitative trading. *ACM Trans Knowl Disc Data* 16(4):1–22. <https://doi.org/10.1145/3488378>
- Liu P, Zhang Y, Bao F et al (2023) Multi-type data fusion framework based on deep reinforcement learning for algorithmic trading. *Appl Intell* 53(2):1683–1706. <https://doi.org/10.1007/s10489-022-03321-w>
- Liu W, Ge Y, Gu Y (2024) Multi-factor stock price prediction based on gan-trellisnet. *Knowl Inf Syst*. <https://doi.org/10.1007/s10115-024-02085-8>
- Ma C, Zhang J, Liu J et al (2021) A parallel multi-module deep reinforcement learning algorithm for stock trading. *Neurocomputing* 449:290–302. <https://doi.org/10.1016/j.neucom.2021.04.005>
- Ma G, Chen P, Liu Z et al (2022) The prediction of enterprise stock change trend by deep neural network model. *Comput Intell Neurosci* 2022:9. <https://doi.org/10.1155/2022/9193055>
- Meng Q (2017) Lightgbm: A highly efficient gradient boosting decision tree. In: *Neural information processing systems*, pp 3149–3157
- Mnih V, Kavukcuoglu K, Silver D et al (2013) Playing atari with deep reinforcement learning. [arXiv:1312.5602](https://arxiv.org/abs/1312.5602)
- Park K, Shin H (2013) Stock price prediction based on a complex interrelation network of economic factors. *Eng Appl Artif Intell* 26(5–6):1550–1561. <https://doi.org/10.1016/j.engappai.2013.01.009>
- Shi Y, Li W, Zhu L et al (2021) Stock trading rule discovery with double deep q-network. *Appl Soft Comput* 107:107320. <https://doi.org/10.1016/j.asoc.2021.107320>
- Soleymani F, Paquet E (2020) Financial portfolio optimization with online deep reinforcement learning and restricted stacked autoencoder-deepbreath. *Expert Syst Appl* 156. <https://doi.org/10.1016/j.eswa.2020.113456>

25. Staffini A (2022) Stock price forecasting by a deep convolutional generative adversarial network. *Front Artif Intell* 5. <https://doi.org/10.3389/frai.2022.837596>
26. Taghian M, Asadi A, Safabakhsh R (2022) Learning financial asset-specific trading rules via deep reinforcement learning. *Expert Syst Appl* 195. <https://doi.org/10.1016/j.eswa.2022.116523>
27. Takara LdA, Santos AAP, Mariani VC et al (2024) Deep reinforcement learning applied to a sparse-reward trading environment with intraday data. *Expert Syst Appl* 238(C). <https://doi.org/10.1016/j.eswa.2023.121897>
28. Touzani Y, Douzi K (2021) An LSTM and GRU based trading strategy adapted to the Moroccan market. *J Big Data* 8(1):126. <https://doi.org/10.1186/s40537-021-00512-z>
29. Wang J, Jing F, He M (2023) Stock trading strategy of reinforcement learning driven by turning point classification. *Neural Process Lett* 55(3, SI):3489–3508. <https://doi.org/10.1007/s11063-022-11019-w>
30. Watkins CJCH (1989) Learning from delayed rewards. PhD thesis, Cambridge University
31. Wu ME, Syu JH, Lin JCW et al (2021) Portfolio management system in equity market neutral using reinforcement learning. *Appl Intell* 51(11):8119–8131. <https://doi.org/10.1007/s10489-021-02262-0>
32. Wu X, Chen H, Wang J et al (2020) Adaptive stock trading strategies with deep reinforcement learning methods. *Inf Sci* 538:142–158. <https://doi.org/10.1016/j.ins.2020.05.066>
33. Yang Z, Zhao T, Wang S et al (2024) Mdf-dmc: A stock prediction model combining multi-view stock data features with dynamic market correlation information. *Expert Syst Appl* 238(E). <https://doi.org/10.1016/j.eswa.2023.122134>
34. Yu X, Li D (2021) Important trading point prediction using a hybrid convolutional recurrent neural network. *Appl Sci-Basel* 11(9):3984. <https://doi.org/10.3390/app11093984>
35. Zhou H, Zhang S, Peng J et al (2021) Informer: Beyond efficient transformer for long sequence time-series forecasting. In: *Proceedings of AAAI*, pp 11106–11115

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Wenjie Liu^{1,2,3} · Yuchen Gu¹  · Yebo Ge¹

Yuchen Gu
guyuchen0820@163.com

Yebo Ge
gdcr0128@126.com

¹ School of Software, Nanjing University of Information Science and Technology, Nanjing 210044, People's Republic of China

² Jiangsu Collaborative Innovation Center of Atmospheric Environment and Equipment Technology (CICAEET), Nanjing 210044, People's Republic of China

³ Jiangsu Province Engineering Research Center of Advanced Computing and Intelligent Services, Nanjing University of Information Science and Technology, Nanjing 210044, People's Republic of China